

Deep Learning – Exam Revision Notes (Part 1)

Self-study based on LectureNotes-5

Contents

1 Study Strategy for 80%+	1
1.1 High-level priorities	1
1.2 Topic-wise weightage intuition	2
1.3 Per-topic routine	2
2 Linear Algebra and Probability Recap	2
2.1 Vector spaces, norms, and orthogonality	2
2.2 Hyperplanes	3
2.3 Singular Value Decomposition (SVD)	3
2.4 Random variables and distributions	3
2.5 Bayes theorem and independence	4
2.6 KL divergence and cross-entropy	4
3 Multilayer Perceptrons (MLPs)	5
3.1 Model structure	5
3.2 Activation functions	5
3.3 Universal approximation intuition	5
3.4 Example numerical – forward pass	5
4 Training Neural Networks	6
4.1 Loss functions	6
4.2 Gradient descent and backpropagation (scalar example)	6
4.3 Backpropagation in a 2-layer MLP	6
4.4 Regularization and generalization	7
5 Convolutional Neural Networks – Core Numericals	7
5.1 Convolution output size	7
5.2 Parameter count	7

1 Study Strategy for 80%+

1.1 High-level priorities

- Master core concepts and simple derivations: linear algebra/probability recap, multilayer perceptrons (MLPs), backpropagation, optimization, and regularization.
- Get very comfortable with numericals around: loss computation, gradient updates, convolution output sizes, parameter counts, and simple probabilistic calculations.
- For every major topic, ensure you can: (i) state key definitions and formulas, (ii) do at least two worked numericals by hand, and (iii) explain intuitively what is happening.

1.2 Topic-wise weightage intuition

- **Foundations (linear algebra, probability, basic ML):** often 15–20% of theory marks through short questions and simple numericals.
- **Feedforward networks training (MLP, backprop, optimization, regularization, normalization):** typically the single largest chunk, 30–40% of theory marks.
- **CNNs, RNNs, attention, autoencoders, modern variants:** together another 40–50%; within this, CNN numericals and LSTM/attention diagrams are high-yield.

1.3 Per-topic routine

For each topic during revision:

1. Read your lecture slides/notes once slowly, highlighting formulas and definitions.
2. Write a one-page summary (bullets + key equations) without looking; then verify and correct.
3. Solve at least 3 numericals: 1 very basic, 1 exam-level, 1 slightly harder.
4. Create a “cheat box”: list of formulas and standard tricks that you can recall in under a minute.

2 Linear Algebra and Probability Recap

2.1 Vector spaces, norms, and orthogonality

Core facts

- A real vector space V is closed under vector addition and scalar multiplication.
- Inner product on $\mathbb{R}^n$: $\langle x, y \rangle = x^\top y$. Orthogonality: $x^\top y = 0$.
- p -norm: $\|x\|_p = (\sum_i |x_i|^p)^{1/p}$, with Euclidean norm $\|x\|_2 = \sqrt{x^\top x}$.

Useful identities

- Projection of x on nonzero v : $\text{proj}_v(x) = \frac{x^\top v}{v^\top v} v$.
- Distance from x to the origin is $\|x\|_2$; distance between x and y is $\|x - y\|_2$.

Example numerical – projection Problem. Let $x = (2, 1)^\top$ and $v = (1, 1)^\top$. Find the projection of x on the line spanned by v .

Solution.

$$\begin{aligned}x^\top v &= 2 \cdot 1 + 1 \cdot 1 = 3, \\v^\top v &= 1^2 + 1^2 = 2, \\\text{proj}_v(x) &= \frac{3}{2}(1, 1)^\top = (1.5, 1.5)^\top.\end{aligned}$$

2.2 Hyperplanes

Core facts

- A (linear) hyperplane in $\mathbb{R}^n$ is an $(n-1)$ -dimensional subspace that passes through the origin.
- A nonzero vector w normal to the hyperplane satisfies $w^\top x = 0$ for all points x on the hyperplane.
- Distance from a point x to the hyperplane $w^\top x = 0$ is $\frac{|w^\top x|}{\|w\|_2}$.

Example numerical – distance to hyperplane Problem. For $w = (1, 2, 2)^\top$ and point $x = (1, 0, 2)^\top$, find the distance from x to the hyperplane $w^\top z = 0$.

Solution.

$$\begin{aligned}w^\top x &= 1 \cdot 1 + 2 \cdot 0 + 2 \cdot 2 = 5, \\ \|w\|_2 &= \sqrt{1^2 + 2^2 + 2^2} = 3, \\ d(x, H) &= \frac{|5|}{3} = \frac{5}{3}.\end{aligned}$$

2.3 Singular Value Decomposition (SVD)

Core facts

- Any $m \times n$ real matrix M admits an SVD: $M = U\Sigma V^\top$, with U and V orthogonal and Σ diagonal with nonnegative singular values.
- Rank- r approximation: keeping only the top r singular values/vectors gives the best rank- r approximation in Frobenius norm.
- Pseudoinverse of rank- r matrix: $M^+ = V\Sigma_r^{-1}U^\top$, where Σ_r^{-1} inverts only nonzero singular values.

Example numerical – low-rank approximation Problem. A 2×2 matrix has singular values $\sigma_1 = 5$, $\sigma_2 = 1$. What fraction of energy is captured by the best rank-1 approximation?

Solution. Energy is $\sigma_1^2 + \sigma_2^2 = 25 + 1 = 26$. Rank-1 energy is $\sigma_1^2 = 25$, so the fraction is $25/26$.

2.4 Random variables and distributions

Core facts

- A random variable is a measurable mapping from sample space Ω to $\mathbb{R}$.
- For discrete r.v., we have a probability mass function (PMF) $p_X(x) = \mathbb{P}(X = x)$.
- For continuous r.v., we have a probability density function (PDF) $f_X(x)$ with $\int f_X(x) dx = 1$.

Gaussian distribution

- Univariate Gaussian: $X \sim \mathcal{N}(\mu, \sigma^2)$ has density

$$f_X(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

- Multivariate Gaussian: $x \sim \mathcal{N}(\mu, \Sigma)$ with density

$$f(x) = \frac{1}{\sqrt{(2\pi)^n \det \Sigma}} \exp\left(-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\right).$$

Example numerical – conditional probability Problem. A data point is positive with probability 0.4. A classifier predicts positive with probability 0.5. If the probability that the classifier predicts positive given that the point is positive is 0.8, what is the probability that the point is positive given that the classifier predicted positive?

Solution.

$$\begin{aligned}\mathbb{P}(Y = 1) &= 0.4, \\ \mathbb{P}(\hat{Y} = 1 \mid Y = 1) &= 0.8, \\ \Rightarrow \mathbb{P}(Y = 1, \hat{Y} = 1) &= 0.4 \times 0.8 = 0.32.\end{aligned}$$

Use total probability for $\hat{Y} = 1$:

$$\mathbb{P}(\hat{Y} = 1) = 0.32 + \mathbb{P}(\hat{Y} = 1 \mid Y = 0) \mathbb{P}(Y = 0).$$

If we are told $\mathbb{P}(\hat{Y} = 1) = 0.5$, then

$$\begin{aligned}0.5 &= 0.32 + \mathbb{P}(\hat{Y} = 1 \mid Y = 0) \times 0.6, \\ \mathbb{P}(\hat{Y} = 1 \mid Y = 0) &= \frac{0.18}{0.6} = 0.3.\end{aligned}$$

Posterior probability is

$$\mathbb{P}(Y = 1 \mid \hat{Y} = 1) = \frac{\mathbb{P}(Y = 1, \hat{Y} = 1)}{\mathbb{P}(\hat{Y} = 1)} = \frac{0.32}{0.5} = 0.64.$$

2.5 Bayes theorem and independence

Core facts

- Bayes theorem: $\mathbb{P}(X \mid Y) = \frac{\mathbb{P}(Y \mid X) \mathbb{P}(X)}{\mathbb{P}(Y)}$.
- Independence: X and Y independent if $\mathbb{P}(X, Y) = \mathbb{P}(X)\mathbb{P}(Y)$, equivalently $\mathbb{P}(X \mid Y) = \mathbb{P}(X)$.

2.6 KL divergence and cross-entropy

Core facts

- KL divergence for discrete distributions: $D_{\text{KL}}(P \parallel Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$.
- Cross-entropy: $H(P; Q) = -\sum_x P(x) \log Q(x) = H(P) + D_{\text{KL}}(P \parallel Q)$.
- Minimizing cross-entropy with respect to Q is equivalent to minimizing KL divergence.

Example numerical – KL for Bernoulli Problem. For Bernoulli distributions $P = \text{Bern}(p)$ and $Q = \text{Bern}(q)$, compute $D_{\text{KL}}(P \parallel Q)$.

Solution.

$$D_{\text{KL}}(P \parallel Q) = p \log \frac{p}{q} + (1 - p) \log \frac{1 - p}{1 - q}.$$

3 Multilayer Perceptrons (MLPs)

3.1 Model structure

Core facts

- An MLP is a composition of affine layers and nonlinear activations.
- For input $x \in \mathbb{R}^{d_0}$, a layer ℓ computes

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \quad a^{(\ell)} = \phi(z^{(\ell)}),$$

where $a^{(0)} = x$ and ϕ is an activation function.

- Output layer often uses identity (regression) or softmax (classification).

3.2 Activation functions

Common choices

- Sigmoid: $\sigma(z) = \frac{1}{1 + e^{-z}}$, derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$; can saturate and cause vanishing gradients.
- Tanh: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, derivative $1 - \tanh^2(z)$; zero-centered but still saturates.
- ReLU: $\text{ReLU}(z) = \max(0, z)$; piecewise linear, alleviates vanishing gradient but can “die” when activations stay negative.

3.3 Universal approximation intuition

Key idea

- A feedforward network with at least one hidden layer and non-polynomial activation (e.g., sigmoid, ReLU) can approximate any continuous function on a compact set to arbitrary accuracy, given enough hidden units.
- Depth can dramatically reduce the number of units required compared to a shallow network for some classes of functions.

3.4 Example numerical – forward pass

Problem. Consider a 2-2-1 MLP with sigmoid activation in the hidden layer and identity in the output. Let

$$\begin{aligned} x &= (x_1, x_2)^\top = (1, 0)^\top, \\ W^{(1)} &= \begin{bmatrix} 1 & -1 \\ 0 & 2 \end{bmatrix}, & b^{(1)} &= \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \\ W^{(2)} &= \begin{bmatrix} 1 & 1 \end{bmatrix}, & b^{(2)} &= 0. \end{aligned}$$

Compute the network output.

Solution.

$$\begin{aligned} z^{(1)} &= W^{(1)}x + b^{(1)} = \begin{bmatrix} 1 & -1 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \\ a^{(1)} &= \sigma(z^{(1)}) = \begin{bmatrix} \sigma(1) \\ \sigma(0) \end{bmatrix} = \begin{bmatrix} \sigma(1) \\ 1/2 \end{bmatrix}, \\ y &= W^{(2)}a^{(1)} + b^{(2)} = 1 \cdot \sigma(1) + 1 \cdot 1/2 = \sigma(1) + 0.5. \end{aligned}$$

4 Training Neural Networks

4.1 Loss functions

Common losses

- Regression: mean squared error (MSE) for scalar output y and prediction $\hat{y}$:

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2.$$

- Binary classification: binary cross-entropy for label $y \in \{0, 1\}$ and predicted probability $\hat{p}$:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)].$$

- Multi-class classification: categorical cross-entropy with softmax outputs.

4.2 Gradient descent and backpropagation (scalar example)

Scalar chain rule If L depends on w through intermediate variable z : $L = L(z(w))$, then

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w}.$$

Backpropagation applies this recursively layer by layer.

Example numerical – one-step gradient descent Problem. For a single-parameter model $\hat{y} = wx$ with loss $L(w) = \frac{1}{2}(y - wx)^2$, compute one step of gradient descent with learning rate η .

Solution.

$$\begin{aligned} L(w) &= \frac{1}{2}(y - wx)^2, \\ \frac{\partial L}{\partial w} &= (y - wx)(-x) = -x(y - wx), \\ w_{\text{new}} &= w - \eta \frac{\partial L}{\partial w} = w + \eta x(y - wx). \end{aligned}$$

4.3 Backpropagation in a 2-layer MLP

Setup For a single data point, consider

$$\begin{aligned} z^{(1)} &= W^{(1)}x + b^{(1)}, & a^{(1)} &= \phi(z^{(1)}), \\ z^{(2)} &= W^{(2)}a^{(1)} + b^{(2)}, & \hat{y} &= z^{(2)}, \\ L &= \frac{1}{2}(\hat{y} - y)^2. \end{aligned}$$

Gradients

$$\begin{aligned} \delta^{(2)} &= \frac{\partial L}{\partial z^{(2)}} = \hat{y} - y, \\ \frac{\partial L}{\partial W^{(2)}} &= \delta^{(2)}(a^{(1)})^\top, \\ \delta^{(1)} &= (W^{(2)})^\top \delta^{(2)} \odot \phi'(z^{(1)}), \\ \frac{\partial L}{\partial W^{(1)}} &= \delta^{(1)}x^\top. \end{aligned}$$

Here $\odot$ denotes elementwise product.

4.4 Regularization and generalization

Core techniques

- **L2 regularization:** add $\lambda\|W\|_2^2$ to the loss; gradient of this term wrt W is $2\lambda W$.
- **Dropout:** randomly drop units during training to reduce co-adaptation; at test time, use full network with scaled weights.
- **Data augmentation:** synthesize additional training examples (e.g., flips, crops) to improve generalization.

Example numerical – L2-regularized update Problem. With loss $L(w) = \frac{1}{2}(y - wx)^2 + \frac{\lambda}{2}w^2$, derive gradient descent update.

Solution.

$$\begin{aligned}\frac{\partial L}{\partial w} &= -x(y - wx) + \lambda w, \\ w_{\text{new}} &= w - \eta[-x(y - wx) + \lambda w] = w + \eta x(y - wx) - \eta \lambda w.\end{aligned}$$

5 Convolutional Neural Networks – Core Numericals

5.1 Convolution output size

Formula For input of spatial size W_{in} , kernel size K , padding P , stride S , the one-dimensional output size is

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} - K + 2P}{S} \right\rfloor + 1.$$

The same applies independently along height and width for 2D convolutions.

Example numerical – no padding, stride 1 Problem. A 5×5 input is convolved with a 3×3 kernel, stride $S = 1$ and padding $P = 0$. What is the output size?

Solution.

$$\begin{aligned}W_{\text{out}} &= \frac{5 - 3 + 2 \cdot 0}{1} + 1 = 3, \\ H_{\text{out}} &= 3. \\ \Rightarrow \text{output size} &= 3 \times 3.\end{aligned}$$

Example numerical – general case Problem. An image of size 128×128 is passed through a convolution layer with kernel 5×5 , stride 2, and padding 2. What is the output spatial size?

Solution.

$$\begin{aligned}W_{\text{out}} &= \left\lfloor \frac{128 - 5 + 2 \cdot 2}{2} \right\rfloor + 1 = \left\lfloor \frac{127}{2} \right\rfloor + 1 = 63 + 1 = 64, \\ H_{\text{out}} &= 64.\end{aligned}$$

5.2 Parameter count

Formula For a 2D convolution layer with C_{in} input channels, C_{out} output channels, and kernel size $K_h \times K_w$, the number of learnable parameters is

$$C_{\text{out}} \times (C_{\text{in}} K_h K_w + 1),$$

where the +1 accounts for one bias per output channel.

Example numerical – parameters in conv layer Problem. For $C_{\text{in}} = 3$, $C_{\text{out}} = 64$, kernel size 3×3 , compute the number of parameters.

Solution.

$$\text{Weights per filter} = 3 \times 3 \times 3 = 27,$$

$$\text{Weights + bias per filter} = 27 + 1 = 28,$$

$$\text{Total parameters} = 64 \times 28 = 1792.$$

Deep Learning – Exam Revision Notes (Part 2)

Self-study based on LectureNotes-5

Contents

1 Recurrent Neural Networks (RNNs)	1
1.1 Motivation and state space view	1
1.2 Simple RNN cell	2
1.3 One-to-one, many-to-one, many-to-many	2
2 Backpropagation Through Time (BPTT)	2
2.1 Unrolling the RNN	2
2.2 Gradient expressions (scalar case)	3
3 Gated Recurrent Unit (GRU)	3
3.1 Equations	3
3.2 Example numerical – GRU step	3
4 Long Short-Term Memory (LSTM)	4
4.1 Equations with cell state	4
4.2 Example numerical – single LSTM step	4
4.3 Parameter count in LSTM	5
5 Stacked and Bidirectional RNNs	5
5.1 Stacked RNNs	5
5.2 Bidirectional RNNs	5
5.3 Example numerical – BiRNN dimensions	5
6 Attention Mechanism	5
6.1 Motivation	5
6.2 Keys, queries, and values	6
6.3 Example numerical – scaled dot-product attention	6
7 Multi-Head Attention and Transformers	6
7.1 Multi-head attention	6
7.2 Example numerical – head dimensionality	7
7.3 Transformer encoder block (high level)	7

1 Recurrent Neural Networks (RNNs)

1.1 Motivation and state space view

Core ideas

- Many problems involve variable-length sequences: time series forecasting, language modelling, speech recognition, translation, etc.

- A state-space view keeps a hidden state h_t summarizing all relevant past information and produces output y_t :

$$h_t = g(x_t, h_{t-1}), \quad y_t = f(h_t).$$

- Parameters of g and f are shared across time steps, allowing the same network to handle sequences of arbitrary length.

1.2 Simple RNN cell

Definition A common parametrization of the recurrent update is

$$h_t = \phi(W_h h_{t-1} + W_x x_t + b_h), \quad y_t = W_y h_t + b_y,$$

where ϕ is typically $\tanh$ or ReLU , $x_t \in \mathbb{R}^{d_x}$, $h_t \in \mathbb{R}^{d_h}$.

Example numerical – forward pass through RNN Problem. Consider a 1D RNN with scalar hidden state and input:

$$\begin{aligned} h_t &= \tanh(0.5 h_{t-1} + x_t), \\ y_t &= h_t. \end{aligned}$$

Let $h_0 = 0$ and inputs be $x_1 = 1$, $x_2 = 0.5$. Compute h_1, h_2, y_1, y_2 .

Solution.

$$\begin{aligned} h_1 &= \tanh(0.5 \cdot 0 + 1) = \tanh(1), \\ y_1 &= h_1 = \tanh(1), \\ h_2 &= \tanh(0.5 h_1 + 0.5) = \tanh(0.5 \tanh(1) + 0.5), \\ y_2 &= h_2. \end{aligned}$$

You should be able to compute these numerically with a calculator in an exam.

1.3 One-to-one, many-to-one, many-to-many

Use cases

- One-to-one: plain feedforward net (non-sequential).
- Many-to-one: sentiment analysis from a sentence, sequence classification; output only after final time step.
- Many-to-many: tagging, translation, speech recognition where a token-level output is produced for each (or many) inputs.

2 Backpropagation Through Time (BPTT)

2.1 Unrolling the RNN

Idea

- Over a finite horizon T , an RNN can be viewed as a deep feedforward network of depth T with shared weights at each layer.
- Backpropagation through time (BPTT) applies the chain rule across all time steps, accumulating gradients through repeated multiplication by W_h and derivatives of ϕ .

2.2 Gradient expressions (scalar case)

Setup For a scalar RNN

$$h_t = \phi(wh_{t-1} + ux_t), \quad y_t = h_t, \quad L = \sum_{t=1}^T \ell_t(y_t),$$

consider the gradient w.r.t. w .

Key recursion Define $\delta_t = \partial L / \partial h_t$. Then

$$\begin{aligned} \delta_t &= \frac{\partial L}{\partial h_t} = \frac{\partial \ell_t}{\partial y_t} \frac{\partial y_t}{\partial h_t} + \delta_{t+1} \frac{\partial h_{t+1}}{\partial h_t} \\ &= \ell'_t(y_t) + \delta_{t+1} \phi'(wh_t + ux_{t+1}) w. \end{aligned}$$

The gradient w.r.t. w is then

$$\frac{\partial L}{\partial w} = \sum_{t=1}^T \delta_t \phi'(wh_{t-1} + ux_t) h_{t-1}.$$

Example numerical – vanishing vs exploding Problem. Suppose $\phi(z) = \tanh(z)$ so that $|\phi'(z)| \leq 1$, and $|w| = 1.5$. Qualitatively, what happens to gradients as T grows?

Answer. The product of many terms of the form $w \phi'(\cdot)$ has magnitude typically larger than 1, so gradients tend to explode as T increases; if instead $|w| < 1$, gradients tend to vanish.

3 Gated Recurrent Unit (GRU)

3.1 Equations

Definition Given input x_t and previous hidden state h_{t-1} , a GRU cell computes

$$\begin{aligned} r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) && \text{(reset gate),} \\ z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) && \text{(update gate),} \\ \tilde{h}_t &= \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h) && \text{(candidate state),} \\ h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t && \text{(hidden state).} \end{aligned}$$

Interpretation

- Reset gate r_t controls how much past state to use in computing the candidate state.
- Update gate z_t interpolates between keeping the old state h_{t-1} and using the new candidate $\tilde{h}_t$.
- This gating helps preserve long-term information by providing nearly-identity paths for gradient flow when $z_t \approx 0$.

3.2 Example numerical – GRU step

Problem. Consider a scalar GRU (all variables are scalars) with

$$\begin{aligned} r_t &= \sigma(2x_t + h_{t-1}), \\ z_t &= \sigma(x_t + h_{t-1}), \\ \tilde{h}_t &= \tanh(x_t + r_t h_{t-1}), \\ h_t &= (1 - z_t) h_{t-1} + z_t \tilde{h}_t. \end{aligned}$$

Let $h_0 = 0$ and input $x_1 = 1$. Compute $r_1, z_1, \tilde{h}_1, h_1$.

Solution.

$$\begin{aligned} r_1 &= \sigma(2 \cdot 1 + 0) = \sigma(2), \\ z_1 &= \sigma(1 + 0) = \sigma(1), \\ \tilde{h}_1 &= \tanh(1 + r_1 \cdot 0) = \tanh(1), \\ h_1 &= (1 - z_1) \cdot 0 + z_1 \tilde{h}_1 = z_1 \tanh(1). \end{aligned}$$

In an exam, you would plug in approximate values $\sigma(1)$, $\sigma(2)$, and $\tanh(1)$.

4 Long Short-Term Memory (LSTM)

4.1 Equations with cell state

Definition An LSTM augments the hidden state h_t with a cell state c_t to better preserve long-term information. One standard formulation is

$$\begin{aligned} f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f) && \text{(forget gate),} \\ i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i) && \text{(input gate),} \\ o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o) && \text{(output gate),} \\ \tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c) && \text{(candidate cell),} \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \\ h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

Interpretation

- Forget gate f_t decides what fraction of the previous cell state to retain.
- Input gate i_t and candidate $\tilde{c}_t$ decide what new information to write into the cell.
- Output gate o_t controls how much of the cell state influences the hidden state.

4.2 Example numerical – single LSTM step

Problem. For a scalar LSTM, suppose

$$\begin{aligned} f_t &= \sigma(1.0 x_t + 0.5 h_{t-1}), \\ i_t &= \sigma(0.5 x_t + 0.5 h_{t-1}), \\ o_t &= \sigma(0.8 x_t + 0.1 h_{t-1}), \\ \tilde{c}_t &= \tanh(0.7 x_t + 0.3 h_{t-1}). \end{aligned}$$

Given $x_t = 1$, $h_{t-1} = 0$, $c_{t-1} = 0.5$, compute $f_t, i_t, o_t, \tilde{c}_t, c_t, h_t$.

Solution.

$$\begin{aligned} f_t &= \sigma(1.0 \cdot 1 + 0.5 \cdot 0) = \sigma(1.0), \\ i_t &= \sigma(0.5 \cdot 1 + 0.5 \cdot 0) = \sigma(0.5), \\ o_t &= \sigma(0.8 \cdot 1 + 0.1 \cdot 0) = \sigma(0.8), \\ \tilde{c}_t &= \tanh(0.7 \cdot 1 + 0.3 \cdot 0) = \tanh(0.7), \\ c_t &= f_t \cdot 0.5 + i_t \cdot \tilde{c}_t, \\ h_t &= o_t \cdot \tanh(c_t). \end{aligned}$$

Again, approximate gate and tanh values numerically.

4.3 Parameter count in LSTM

Formula For input dimension d_x and hidden size d_h , an LSTM has four sets of weights and biases (for f, i, o, c). The parameter count is

$$4(d_h d_x + d_h^2 + d_h).$$

Example numerical – LSTM parameters Problem. An LSTM has input size $d_x = 20$ and hidden size $d_h = 50$. How many learnable parameters does the LSTM cell have?

Solution.

$$\begin{aligned}\text{Parameters} &= 4(50 \cdot 20 + 50^2 + 50) \\ &= 4(1000 + 2500 + 50) \\ &= 4 \cdot 3550 = 14200.\end{aligned}$$

5 Stacked and Bidirectional RNNs

5.1 Stacked RNNs

Definition

- In a stacked RNN, multiple recurrent layers are placed on top of each other.
- For layer ℓ :

$$h_t^{(\ell)} = g^{(\ell)}(h_t^{(\ell-1)}, h_{t-1}^{(\ell)}), \quad h_t^{(0)} = x_t.$$

- Higher layers can capture longer-term and more abstract patterns.

5.2 Bidirectional RNNs

Idea

- When the entire input sequence is known, a bidirectional RNN (BiRNN) processes the sequence in both forward and backward directions.
- Forward state $\vec{h}_t$ runs from 1 to T ; backward state $\overleftarrow{h}_t$ runs from T to 1.
- The combined state at time t is typically $[\vec{h}_t; \overleftarrow{h}_t]$, which is then fed to an output layer.

5.3 Example numerical – BiRNN dimensions

Problem. A bidirectional RNN has hidden size $d_h = 32$ in each direction. What is the dimension of the concatenated hidden state and of the output if a linear layer maps this state to a 10-class softmax?

Solution.

- Concatenated hidden state has dimension $2d_h = 64$.
- The output layer weight matrix has shape 10×64 , so it has $10 \cdot 64 = 640$ weights plus 10 biases.

6 Attention Mechanism

6.1 Motivation

Key question Instead of compressing all past information into a single vector h_t , attention lets the model look back at the entire sequence of inputs (or encoded states) and weight them according to their relevance to the current query.

6.2 Keys, queries, and values

Definitions Given input sequence vectors $x_1, \dots, x_N$, compute for each position

$$\begin{aligned} k_i &= W_k x_i && \text{(key),} \\ v_i &= W_v x_i && \text{(value),} \\ q &= W_q h && \text{(query from some state } h\text{).} \end{aligned}$$

The attention score for position i is

$$e_i = \frac{q^\top k_i}{\sqrt{d_k}}, \quad \alpha_i = \frac{\exp(e_i)}{\sum_j \exp(e_j)},$$

and the context vector is

$$c = \sum_{i=1}^N \alpha_i v_i.$$

6.3 Example numerical – scaled dot-product attention

Problem. Suppose the query and two keys are

$$\begin{aligned} q &= (1, 0)^\top, \\ k_1 &= (1, 1)^\top, \\ k_2 &= (0, 1)^\top, \end{aligned}$$

with $d_k = 2$, and values $v_1 = 2, v_2 = 0$ (scalars). Compute attention weights α_1, α_2 and context c .

Solution.

$$\begin{aligned} e_1 &= \frac{q^\top k_1}{\sqrt{2}} = \frac{1 \cdot 1 + 0 \cdot 1}{\sqrt{2}} = \frac{1}{\sqrt{2}}, \\ e_2 &= \frac{q^\top k_2}{\sqrt{2}} = \frac{1 \cdot 0 + 0 \cdot 1}{\sqrt{2}} = 0. \end{aligned}$$

Softmax gives

$$\begin{aligned} \alpha_1 &= \frac{\exp(e_1)}{\exp(e_1) + \exp(e_2)}, \\ \alpha_2 &= \frac{\exp(e_2)}{\exp(e_1) + \exp(e_2)} = 1 - \alpha_1. \end{aligned}$$

The context is

$$c = \alpha_1 v_1 + \alpha_2 v_2 = 2\alpha_1.$$

7 Multi-Head Attention and Transformers

7.1 Multi-head attention

Idea

- Instead of using a single query-key-value projection, multi-head attention uses multiple sets of projections to capture different types of relationships.
- For each head j :

$$Q_j = XW_j^Q, \quad K_j = XW_j^K, \quad V_j = XW_j^V,$$

apply scaled dot-product attention separately and then concatenate the head outputs.

7.2 Example numerical – head dimensionality

Problem. An attention layer uses model dimension $d_{\text{model}} = 128$ and $h = 8$ heads. What is the per-head key/query dimension if they are split evenly? What is the shape of W^Q if it maps from d_{model} to all heads at once?

Solution.

- Per-head dimension is $d_k = d_{\text{model}}/h = 128/8 = 16$.
- If $Q = XW^Q$ contains all heads concatenated, W^Q has shape $d_{\text{model}} \times (hd_k) = 128 \times 128$.

7.3 Transformer encoder block (high level)

Structure A typical encoder block consists of:

1. Multi-head self-attention with residual connection and layer normalization.
2. Position-wise feedforward network (two linear layers with nonlinearity) with another residual connection and layer normalization.

Positional encoding

- Because self-attention is permutation-invariant, positional encodings p_t are added to token embeddings x_t so that $\tilde{x}_t = x_t + p_t$.
- Typical exam questions ask for the purpose of positional encoding and how it allows the transformer to be sensitive to order.

Deep Learning – Exam Revision Notes (Part 3)

Self-study based on LectureNotes-5

Contents

1 Unsupervised Learning and Representation	1
1.1 Supervised vs unsupervised	1
1.2 Representation learning	2
2 Basic Autoencoder	2
2.1 Architecture	2
2.2 Reconstruction loss	2
2.3 Example numerical – reconstruction error	2
3 Connection to PCA	2
3.1 Linear autoencoder and PCA	2
3.2 Example numerical – variance captured	3
4 Autoencoder Variants	3
4.1 Denoising autoencoder (DAE)	3
4.2 Sparse autoencoder	3
4.3 Contractive autoencoder	3
5 Variational Autoencoder (VAE)	4
5.1 Generative view	4
5.2 Objective (ELBO)	4
5.3 Gaussian encoder and KL term	4
5.4 Example numerical – VAE KL term	4
5.5 Reparameterization trick	4
6 State-of-the-Art (Very Brief Exam Notes)	5
6.1 Beyond basic autoencoders	5

1 Unsupervised Learning and Representation

1.1 Supervised vs unsupervised

Core ideas

- Supervised learning uses labeled data (x, y) and optimizes a loss between model outputs and labels (e.g., cross-entropy, MSE).
- Unsupervised learning deals with unlabeled data x and aims to learn structure in the data: clustering, density estimation, dimensionality reduction, or feature learning.
- Autoencoders are neural architectures for unsupervised representation learning via reconstruction through a low-dimensional bottleneck.

1.2 Representation learning

Goals

- Learn a mapping from inputs to feature vectors (embeddings) that capture meaningful structure.
- Good representations make downstream tasks (classification, regression, generation) easier with simple models.
- Autoencoders, VAEs, and many generative models fit into this representation-learning view.

2 Basic Autoencoder

2.1 Architecture

Encoder-decoder

- Encoder: $z = f_{\theta}(x)$, maps input $x \in \mathbb{R}^d$ to latent code $z \in \mathbb{R}^k$.
- Decoder: $\hat{x} = g_{\phi}(z)$, attempts to reconstruct x from z .
- Bottleneck: $k < d$ (undercomplete) forces network to compress information; with nonlinearity, this learns nonlinear manifolds.

2.2 Reconstruction loss

Typical objectives For continuous inputs, a common loss is mean squared error (MSE):

$$L_{\text{AE}} = \frac{1}{N} \sum_{i=1}^N \|x^{(i)} - \hat{x}^{(i)}\|_2^2.$$

For binary or bounded inputs, cross-entropy can be used with a sigmoid output.

2.3 Example numerical – reconstruction error

Problem. Suppose an autoencoder reconstructs a 3D input $x = (1, 0, -1)^{\top}$ as $\hat{x} = (0.8, 0.2, -0.9)^{\top}$. Compute the squared reconstruction error and the MSE over this single sample.

Solution.

$$\begin{aligned} x - \hat{x} &= (0.2, -0.2, -0.1)^{\top}, \\ \|x - \hat{x}\|_2^2 &= 0.2^2 + (-0.2)^2 + (-0.1)^2 = 0.04 + 0.04 + 0.01 = 0.09, \\ \text{MSE (single sample)} &= 0.09. \end{aligned}$$

3 Connection to PCA

3.1 Linear autoencoder and PCA

Key fact

- If the encoder and decoder are linear (no activation), the loss is MSE, and the latent dimension is k , then the optimal solution spans the same subspace as the top k principal components of the data.
- PCA can be seen as a special case of an autoencoder with encoder $E(X) = W^{\top}X$ and decoder $D(E(X)) = WW^{\top}X$ for an appropriate orthonormal matrix W .

3.2 Example numerical – variance captured

Problem. Data has principal component eigenvalues $\lambda_1 = 5, \lambda_2 = 2, \lambda_3 = 1$. If an undercomplete linear autoencoder uses a 2D bottleneck corresponding to the top two components, what fraction of total variance is captured?

Solution.

$$\begin{aligned}\text{Total variance} &= 5 + 2 + 1 = 8, \\ \text{Variance in top-2} &= 5 + 2 = 7, \\ \text{Fraction captured} &= 7/8.\end{aligned}$$

4 Autoencoder Variants

4.1 Denoising autoencoder (DAE)

Idea

- Corrupt input with noise or masking: $\tilde{x} = x + \varepsilon$ or randomly zero out components.
- Encoder sees only $\tilde{x}$, decoder tries to reconstruct clean x .
- Objective: $L_{\text{DAE}} = \frac{1}{N} \sum_i \|x^{(i)} - g_\phi(f_\theta(\tilde{x}^{(i)}))\|^2$.
- This encourages robustness to noise and prevents trivial identity mapping.

Example numerical – noisy input Problem. Let $x = (1, 1)^\top$ and noise $\varepsilon \sim \mathcal{N}(0, 0.1^2 I)$. One corrupted sample is $\tilde{x} = (0.9, 1.1)^\top$, and the autoencoder reconstructs $\hat{x} = (0.95, 0.98)^\top$. Compute the squared reconstruction error.

Solution.

$$\begin{aligned}x - \hat{x} &= (0.05, 0.02)^\top, \\ \|x - \hat{x}\|_2^2 &= 0.05^2 + 0.02^2 = 0.0025 + 0.0004 = 0.0029.\end{aligned}$$

4.2 Sparse autoencoder

Idea

- Encourage most hidden units to be near zero for each input: promotes a sparse code.
- Typical loss: $L = L_{\text{recon}} + \lambda \sum_j R(\hat{\rho}_j)$, where $\hat{\rho}_j$ is average activation of unit j and R penalizes deviation from a small target sparsity ρ .
- A common choice is KL divergence: $R(\hat{\rho}_j) = \rho \log \frac{\rho}{\hat{\rho}_j} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_j}$.

4.3 Contractive autoencoder

Idea

- Penalize sensitivity of the encoder to input perturbations by adding the Jacobian norm $\|\partial h / \partial x\|_F^2$ to the loss.
- This encourages locally smooth, robust representations around the data manifold.

5 Variational Autoencoder (VAE)

5.1 Generative view

Latent variable model

- Assume latent variable $z \sim p(z)$, typically $\mathcal{N}(0, I)$.
- Decoder defines $p_\theta(x | z)$; sampling: $z \sim p(z)$, then $x \sim p_\theta(x | z)$.
- Inference is intractable, so VAE introduces an encoder $q_\phi(z | x)$ to approximate the posterior.

5.2 Objective (ELBO)

Evidence lower bound The VAE maximizes, for each data point x , the ELBO

$$\mathcal{L}_{\text{ELBO}}(x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x | z)] - D_{\text{KL}}(q_\phi(z | x) \| p(z)).$$

This can be interpreted as reconstruction term minus a regularization term that keeps $q_\phi(z | x)$ close to $p(z)$.

5.3 Gaussian encoder and KL term

Form of $q_\phi(z | x)$

- Typically, $q_\phi(z | x)$ is chosen Gaussian: $\mathcal{N}(\mu(x), \text{diag}(\sigma^2(x)))$.
- With standard normal prior $p(z) = \mathcal{N}(0, I)$, the KL divergence can be computed in closed form.

KL divergence for 1D Gaussian For $q(z) = \mathcal{N}(\mu, \sigma^2)$ and $p(z) = \mathcal{N}(0, 1)$,

$$D_{\text{KL}}(q \| p) = \frac{1}{2}(-\log \sigma^2 - 1 + \sigma^2 + \mu^2).$$

5.4 Example numerical – VAE KL term

Problem. For a 1D latent with $\mu = 1$ and $\sigma^2 = 0.25$, compute $D_{\text{KL}}(q \| p)$.

Solution.

$$\begin{aligned} D_{\text{KL}}(q \| p) &= \frac{1}{2}(-\log 0.25 - 1 + 0.25 + 1^2) \\ &= \frac{1}{2}(-(-1.3863) - 1 + 0.25 + 1) \\ &\approx \frac{1}{2}(1.3863 + 0.25) = \frac{1}{2}(1.6363) \approx 0.8182. \end{aligned}$$

(Using $\log 0.25 \approx -1.3863$.)

5.5 Reparameterization trick

Gradient-friendly sampling

- Directly sampling $z \sim q_\phi(z | x)$ makes gradients w.r.t. ϕ hard to compute.
- Reparameterize: $z = \mu(x) + \sigma(x) \odot \varepsilon$, where $\varepsilon \sim \mathcal{N}(0, I)$ is noise independent of ϕ .
- This allows gradients to flow through $\mu(x)$ and $\sigma(x)$ via standard backpropagation.

6 State-of-the-Art (Very Brief Exam Notes)

6.1 Beyond basic autoencoders

Examples

- **Graph neural networks (GNNs):** operate on graph-structured data, aggregating node and neighbor features.
- **Physics-informed neural networks (PINNs):** incorporate differential equation constraints into the loss.
- **Diffusion and generative models:** iteratively transform noise into data using learned denoising steps.

Exam-focus tip

- For this course, expect at most short-answer conceptual questions on these topics, such as definitions, high-level architectures, or use-cases.
- Main numericals will more likely center on autoencoders (reconstruction error, bottleneck sizes, parameter counts) and VAEs (KL term, ELBO components).

Exam-style Numericals Across the Syllabus

1. Linear Algebra / Geometry

Problem 1 (Projection and distance to hyperplane). Let $x = (2, 1, 2)^\top$ and let the hyperplane in $\mathbb{R}^3$ be given by

$$w^\top z = 0, \quad w = (1, 2, 2)^\top.$$

(a) Find the projection of x onto the line spanned by w . (b) Compute the distance from x to the hyperplane.

Solution. (a) Projection of x onto the direction w is

$$\text{proj}_w(x) = \frac{x^\top w}{w^\top w} w.$$

Compute inner products:

$$x^\top w = 2 \cdot 1 + 1 \cdot 2 + 2 \cdot 2 = 2 + 2 + 4 = 8, \quad w^\top w = 1^2 + 2^2 + 2^2 = 1 + 4 + 4 = 9.$$

So

$$\text{proj}_w(x) = \frac{8}{9}(1, 2, 2)^\top = \left(\frac{8}{9}, \frac{16}{9}, \frac{16}{9}\right)^\top.$$

(b) Distance from x to the hyperplane $w^\top z = 0$ is

$$d(x, H) = \frac{|w^\top x|}{\|w\|_2} = \frac{|8|}{\sqrt{9}} = \frac{8}{3}.$$

2. Probability / Bayes

Problem 2 (Posterior using Bayes theorem). A binary classifier predicts $\hat{Y} \in \{0, 1\}$ for true label $Y \in \{0, 1\}$. Suppose

$$\mathbb{P}(Y = 1) = 0.4, \quad \mathbb{P}(\hat{Y} = 1 \mid Y = 1) = 0.9, \quad \mathbb{P}(\hat{Y} = 1 \mid Y = 0) = 0.2.$$

Compute $\mathbb{P}(Y = 1 \mid \hat{Y} = 1)$.

Solution. Joint probabilities:

$$\mathbb{P}(Y = 1, \hat{Y} = 1) = 0.4 \cdot 0.9 = 0.36,$$

$$\mathbb{P}(Y = 0, \hat{Y} = 1) = 0.6 \cdot 0.2 = 0.12.$$

Total probability that classifier predicts positive:

$$\mathbb{P}(\hat{Y} = 1) = 0.36 + 0.12 = 0.48.$$

Bayes theorem:

$$\mathbb{P}(Y = 1 \mid \hat{Y} = 1) = \frac{\mathbb{P}(Y = 1, \hat{Y} = 1)}{\mathbb{P}(\hat{Y} = 1)} = \frac{0.36}{0.48} = 0.75.$$

3. Single-parameter Gradient Descent

Problem 3 (One-step GD update). Consider model $\hat{y} = wx$ with loss

$$L(w) = \frac{1}{2}(y - wx)^2.$$

Given a single training point $(x, y) = (2, 3)$ and current parameter $w^{(0)} = 1$, compute one gradient descent step with learning rate $\eta = 0.1$.

Solution. Gradient:

$$\frac{\partial L}{\partial w} = -x(y - wx).$$

At $w^{(0)} = 1$:

$$y - w^{(0)}x = 3 - 1 \cdot 2 = 1, \quad \left. \frac{\partial L}{\partial w} \right|_{w^{(0)}} = -2 \cdot 1 = -2.$$

Update:

$$w^{(1)} = w^{(0)} - \eta \frac{\partial L}{\partial w} = 1 - 0.1 \cdot (-2) = 1 + 0.2 = 1.2.$$

4. MLP Forward and Backward

Problem 4 (2–2–1 MLP: forward pass and weight gradient). Network:

$$z^{(1)} = W^{(1)}x + b^{(1)}, \quad a^{(1)} = \sigma(z^{(1)}),$$

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}, \quad \hat{y} = z^{(2)},$$

$$L = \frac{1}{2}(\hat{y} - y)^2,$$

where $\sigma(\cdot)$ is sigmoid. Take

$$x = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad W^{(1)} = \begin{bmatrix} 1 & -1 \\ 0 & 2 \end{bmatrix}, \quad b^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad W^{(2)} = \begin{bmatrix} 1 & 1 \end{bmatrix}, \quad b^{(2)} = 0, \quad y = 1.$$

(a) Compute $\hat{y}$. (b) Compute $\partial L / \partial W^{(2)}$.

Solution. Forward:

$$z^{(1)} = W^{(1)}x + b^{(1)} = \begin{bmatrix} 1 & -1 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

$$a^{(1)} = \sigma(z^{(1)}) = \begin{bmatrix} \sigma(1) \\ \sigma(0) \end{bmatrix} = \begin{bmatrix} \sigma(1) \\ 0.5 \end{bmatrix}.$$

Output:

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} = 1 \cdot \sigma(1) + 1 \cdot 0.5 = \sigma(1) + 0.5,$$

so $\hat{y} = \sigma(1) + 0.5$.

Loss:

$$L = \frac{1}{2}(\hat{y} - y)^2.$$

Gradient at output:

$$\frac{\partial L}{\partial z^{(2)}} = \hat{y} - y.$$

Since $z^{(2)}$ is linear in $W^{(2)}$:

$$\frac{\partial L}{\partial W^{(2)}} = \frac{\partial L}{\partial z^{(2)}} (a^{(1)})^\top = (\hat{y} - y) [\sigma(1) \quad 0.5].$$

5. CNN Shapes and Parameters

Problem 5 (Conv + pool shapes and parameter count). Input image: $32 \times 32 \times 3$. Layer 1: Conv with kernel 5×5 , stride 1, padding 2, $C_{\text{out}} = 16$. Layer 2: 2×2 max-pooling with stride 2.

(a) Find output shape after Layer 1. (b) Find output shape after Layer 2. (c) Compute the number of parameters in Layer 1.

Solution. (a) For each spatial dimension,

$$W_{\text{out}} = \frac{W_{\text{in}} - K + 2P}{S} + 1 = \frac{32 - 5 + 2 \cdot 2}{1} + 1 = \frac{31}{1} + 1 = 32.$$

So output is 32×32 with 16 channels: $32 \times 32 \times 16$.

(b) Max-pooling 2×2 with stride 2 halves each spatial dimension:

$$32 \rightarrow 16,$$

so output is $16 \times 16 \times 16$.

(c) Parameters in a conv layer:

$$\text{params} = C_{\text{out}}(C_{\text{in}}K_hK_w + 1) = 16(3 \cdot 5 \cdot 5 + 1) = 16(75 + 1) = 16 \cdot 76 = 1216.$$

6. Simple RNN with BPTT (Short Horizon)

Problem 6 (2-step RNN gradient wrt recurrent weight). Scalar RNN:

$$h_t = \tanh(wh_{t-1} + ux_t), \quad y_t = h_t, \quad L = \frac{1}{2}(y_2 - y^*)^2,$$

with $h_0 = 0$, $x_1 = 1$, $x_2 = 0.5$, target $y^* = 0$. Take $w = 1$, $u = 1$. Compute $\partial L / \partial w$.

Solution. Forward:

$$h_1 = \tanh(wh_0 + ux_1) = \tanh(1 \cdot 0 + 1 \cdot 1) = \tanh(1).$$

$$h_2 = \tanh(wh_1 + ux_2) = \tanh(1 \cdot \tanh(1) + 1 \cdot 0.5) = \tanh(\tanh(1) + 0.5).$$

So $y_2 = h_2$, and

$$L = \frac{1}{2}(h_2 - 0)^2 = \frac{1}{2}h_2^2.$$

Backward:

First,

$$\frac{\partial L}{\partial h_2} = h_2.$$

Derivative of $\tanh$: $\tanh'(z) = 1 - \tanh^2(z)$.

Let $a_2 = wh_1 + ux_2$, so $h_2 = \tanh(a_2)$:

$$\frac{\partial h_2}{\partial a_2} = 1 - h_2^2.$$

Also,

$$\frac{\partial a_2}{\partial w} = h_1, \quad \frac{\partial a_2}{\partial h_1} = w = 1.$$

Gradient wrt w has two paths (through both time steps). Use chain rule via h_1 :

First compute $\partial L / \partial h_1$:

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_2} \frac{\partial h_2}{\partial a_2} \frac{\partial a_2}{\partial h_1} = h_2(1 - h_2^2) \cdot 1.$$

At time $t = 2$, direct contribution to $\partial L / \partial w$:

$$\left. \frac{\partial L}{\partial w} \right|_{t=2} = \frac{\partial L}{\partial h_2} \frac{\partial h_2}{\partial a_2} \frac{\partial a_2}{\partial w} = h_2(1 - h_2^2)h_1.$$

At time $t = 1$, $a_1 = wh_0 + ux_1 = x_1$, $h_1 = \tanh(a_1)$:

$$\frac{\partial h_1}{\partial a_1} = 1 - h_1^2, \quad \frac{\partial a_1}{\partial w} = h_0 = 0.$$

So

$$\left. \frac{\partial L}{\partial w} \right|_{t=1} = \frac{\partial L}{\partial h_1} \frac{\partial h_1}{\partial a_1} \frac{\partial a_1}{\partial w} = \frac{\partial L}{\partial h_1} (1 - h_1^2) \cdot 0 = 0.$$

Total:

$$\frac{\partial L}{\partial w} = h_2(1 - h_2^2)h_1.$$

(You can plug numeric values for $\tanh(1)$ and $\tanh(\tanh(1) + 0.5)$ if needed.)

7. LSTM Single-step Gates

Problem 7 (Compute LSTM gates and states). Scalar LSTM:

$$\begin{aligned} f_t &= \sigma(1.0 x_t + 0.5 h_{t-1}), \\ i_t &= \sigma(0.5 x_t + 0.5 h_{t-1}), \\ o_t &= \sigma(0.8 x_t + 0.1 h_{t-1}), \\ \tilde{c}_t &= \tanh(0.7 x_t + 0.3 h_{t-1}), \\ c_t &= f_t c_{t-1} + i_t \tilde{c}_t, \\ h_t &= o_t \tanh(c_t). \end{aligned}$$

Let $x_t = 1$, $h_{t-1} = 0$, $c_{t-1} = 0.5$. Compute $f_t, i_t, o_t, \tilde{c}_t, c_t, h_t$ symbolically in terms of $\sigma(1), \sigma(0.5), \sigma(0.8), \tanh(0.7)$.

Solution.

$$f_t = \sigma(1.0 \cdot 1 + 0.5 \cdot 0) = \sigma(1),$$

$$i_t = \sigma(0.5 \cdot 1 + 0.5 \cdot 0) = \sigma(0.5),$$

$$o_t = \sigma(0.8 \cdot 1 + 0.1 \cdot 0) = \sigma(0.8),$$

$$\tilde{c}_t = \tanh(0.7 \cdot 1 + 0.3 \cdot 0) = \tanh(0.7).$$

Cell state:

$$c_t = f_t c_{t-1} + i_t \tilde{c}_t = \sigma(1) \cdot 0.5 + \sigma(0.5) \tanh(0.7).$$

Hidden state:

$$h_t = o_t \tanh(c_t) = \sigma(0.8) \tanh(\sigma(1) \cdot 0.5 + \sigma(0.5) \tanh(0.7)).$$

8. Scaled Dot-product Attention

Problem 8 (Two-position attention). Let

$$q = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad k_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad k_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

dimension $d_k = 2$, and scalar values $v_1 = 2, v_2 = 0$. Compute attention weights α_1, α_2 and context c using scaled dot-product attention:

$$e_i = \frac{q^\top k_i}{\sqrt{d_k}}, \quad \alpha_i = \frac{\exp(e_i)}{\sum_j \exp(e_j)}, \quad c = \sum_i \alpha_i v_i.$$

Solution. Scores:

$$e_1 = \frac{q^\top k_1}{\sqrt{2}} = \frac{1 \cdot 1 + 0 \cdot 1}{\sqrt{2}} = \frac{1}{\sqrt{2}},$$

$$e_2 = \frac{q^\top k_2}{\sqrt{2}} = \frac{1 \cdot 0 + 0 \cdot 1}{\sqrt{2}} = 0.$$

Softmax:

$$\alpha_1 = \frac{\exp(e_1)}{\exp(e_1) + \exp(e_2)}, \quad \alpha_2 = \frac{\exp(e_2)}{\exp(e_1) + \exp(e_2)} = 1 - \alpha_1.$$

Context:

$$c = \alpha_1 v_1 + \alpha_2 v_2 = 2\alpha_1.$$

(You can plug numerical values if needed.)

9. Autoencoder / PCA Numericals

Problem 9 (Reconstruction error and variance captured). (a) An autoencoder reconstructs $x = (1, 0, -1)^\top$ as $\hat{x} = (0.8, 0.1, -0.9)^\top$. Compute squared reconstruction error and MSE for this single sample.

(b) Data covariance has eigenvalues $\lambda_1 = 5, \lambda_2 = 2, \lambda_3 = 1$. If a linear autoencoder uses a 2D bottleneck aligned with top two eigenvectors, what fraction of total variance is captured?

Solution. (a) Difference:

$$x - \hat{x} = (0.2, -0.1, -0.1)^\top.$$

Squared error:

$$\|x - \hat{x}\|_2^2 = 0.2^2 + (-0.1)^2 + (-0.1)^2 = 0.04 + 0.01 + 0.01 = 0.06.$$

For one sample, $\text{MSE} = 0.06$.

(b) Total variance:

$$\lambda_1 + \lambda_2 + \lambda_3 = 5 + 2 + 1 = 8.$$

Top-2 variance:

$$5 + 2 = 7.$$

Fraction captured:

$$7/8 = 0.875.$$

10. VAE KL Term

Problem 10 (1D Gaussian KL). In a VAE, suppose $q(z \mid x) = \mathcal{N}(\mu, \sigma^2)$ with $\mu = 1$, $\sigma^2 = 0.25$, and prior $p(z) = \mathcal{N}(0, 1)$. Compute

$$D_{\text{KL}}(q\|p) = \frac{1}{2}(-\log \sigma^2 - 1 + \sigma^2 + \mu^2).$$

Solution. Plug in:

$$D_{\text{KL}}(q\|p) = \frac{1}{2}(-\log 0.25 - 1 + 0.25 + 1^2) = \frac{1}{2}(-\log 0.25 + 0.25).$$

Since $\log 0.25 = \log(1/4) = -\log 4 \approx -1.3863$,

$$-\log 0.25 \approx 1.3863.$$

Thus

$$D_{\text{KL}}(q\|p) \approx \frac{1}{2}(1.3863 + 0.25) = \frac{1}{2} \cdot 1.6363 \approx 0.8182.$$